

Diverse Policy Learning via Random Obstacle Deployment for Zero-Shot Adaptation

Seokjin Choi , Yonghyeon Lee , Seungyeon Kim , Che-Sang Park , Himchan Hwang , and Frank C. Park 

Abstract—In this letter, we propose a novel reinforcement learning framework that enables zero-shot policy adaptation in environments with unseen, dynamically changing obstacles. Adopting the idea that learning a policy capable of generating diverse actions is key to achieving such adaptability, our primary contribution is a novel learning algorithm that incorporates random obstacle deployment, enabling the policy to explore and learn diverse actions. This method overcomes the limitations of existing diverse policy learning approaches, which primarily rely on mutual information maximization to increase diversity. To enable zero-shot dynamic adaptation, our method further involves two key components: a state-dependent latent skill sampler and a motion predictor. We sample multiple skill variables at each state using a skill sampler, then filter out unsafe skills using a motion predictor, and consequently execute actions corresponding to safe skills. Compared to existing methods, our experiments demonstrate that proposed method generates significantly more diverse actions and adapts better to dynamically changing environments, making it highly effective for tasks with varying constraints such as moving obstacles.

Index Terms—Reinforcement learning, motion and path planning.

I. INTRODUCTION

CONSIDER the problem of designing a policy that outputs an end-effector's trajectory to push a T-shaped block into a desired pose as illustrated in Fig. 1 (Upper-Left). We argue

Received 9 September 2024; accepted 6 January 2025. Date of publication 20 January 2025; date of current version 3 February 2025. This article was recommended for publication by Associate Editor F. Zhang and Editor J. Kober upon evaluation of the reviewers' comments. This work was supported in part by IITP-MSIT under Grant RS-2021-II212068 (SNU AI Innovation Hub), in part by IITP-MSIT under Grant 2022-220480 and Grant RS-2022-II220480 (Development of Training and Inference Methods for Goal-Oriented Artificial Intelligence Agents), in part by Ministry of Science, ICT (MSIT), Korea, through Global Research Support Program in the Digital Field program under Grant RS-2024-00436680 supervised by the Institute for Information & Communications Technology Planning & Evaluation (IITP), in part KIAT under Grant P0020536 (HRD Program for Industrial Innovation), in part by SRRC NRF under Grant RS-2023-00208052, in part by SNU-AIIS, in part by SNU-IPAI, in part by SNU-IAMD, in part by SNU BK21+ Program in Mechanical Engineering, in part by the SNU Institute for Engineering Research, in part by Microsoft Research Asia, in part by Hyundai Motor Company, in part by Kia, and in part by a KIAS Individual under Grant AP092701 through the Center for AI and Natural Sciences at Korea Institute for Advanced Study. (Seokjin Choi and Yonghyeon Lee are co-first authors.) (Corresponding author: Frank C. Park.)

Seokjin Choi, Seungyeon Kim, Che-Sang Park, Himchan Hwang, and Frank C. Park are with the Robotics Laboratory, Seoul National University, Seoul 08826, South Korea (e-mail: csj@robotics.snu.ac.kr; ksy@robotics.snu.ac.kr; cspark@robotics.snu.ac.kr; himchan@robotics.snu.ac.kr; fcp@snu.ac.kr).

Yonghyeon Lee is with the Center for AI and Natural Sciences (CAINS), Korea Institute for Advanced Study (KIAS), Seoul 02455, South Korea (e-mail: ylee@kias.re.kr).

This letter has supplementary downloadable material available at <https://doi.org/10.1109/LRA.2025.3532162>, provided by the authors.

Digital Object Identifier 10.1109/LRA.2025.3532162

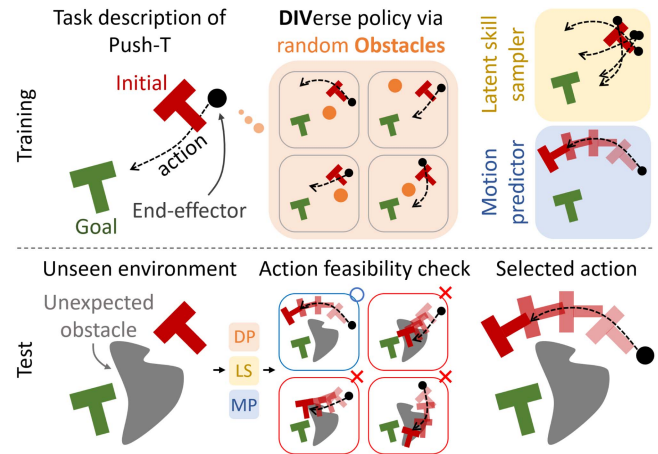


Fig. 1. Push-T task: End-effector is required to push the T-block into a goal pose. Upper: Training of diverse policy, latent skill sampler, and motion predictor. Lower: Adaptation in unseen environments.

that, for this policy to be truly useful, it should be adaptable to a variety of unforeseen obstacles. For example, the policy should work even when unexpected obstacles block certain areas (Fig. 1, Lower-Left), and it should be applicable to dynamically changing environments that involve moving obstacles. Therefore, the problem we address in this paper is learning a policy that enables *zero-shot adaptation to environments with unseen moving obstacles*.

We adopt the idea that a policy capable of generating diverse actions – which we refer to as a *diverse policy* – is key to achieving high adaptability [1], [2], [3]. The more diverse actions a policy can generate, the higher the likelihood that one of these actions will successfully navigate around unforeseen obstacles in a new environment, allowing the selection of an appropriate movement to avoid collisions.

There have been both Learning from Demonstration (LfD) and Reinforcement Learning (RL) approaches in this regard. While LfD is easier to train, the policy's diversity is fundamentally limited by the diversity of the demonstration data, making the data collection process crucial and often costly [2], [3], [4], [5], [6]. On the other hand, RL methods may take longer and be more difficult to train, but they have the potential to learn diverse policies without requiring extensive data, provided an effective diversity-encouraging algorithm is used. In this paper, we focus on the latter approach with the aim of developing such an effective algorithm.

A common feature of the existing RL approaches for diverse policy learning [1], [7], [8], [9], [10] is that the policy is modeled to be dependent on a latent skill variable z . For instance, a policy is defined as a distribution of actions conditioned on a state

s and skill z , i.e., $\pi(a|s, z)$. While maximizing rewards, they aim to maximize the mutual information between the state s or state-action pair (s, a) and the skill variable z .

However, the resulting diversity is far less than desirable, leading to unsatisfactory adaptability as demonstrated in our experiments (Section V). Maximizing mutual information [11], [12] encourages the policy $\pi(a|s, z)$ to vary with z but does not inherently drive large state changes. Instead, it is maximized as long as states are distinct enough to infer the skill, prioritizing distinguishability with respect to z . As noted in [13], [14], this can lead to minimal or static behaviors, restricting the policy's exploration of a broader state or state-action space. Another major limitation is their inability to adapt to dynamically changing environments. This is because they fix the latent skill variable within an episode, preventing the policy from adjusting to new constraints online. If the latent skill is changed mid-episode, the current state might be an outlier for the new latent skill, potentially leading to poor performance.

In this paper, we propose a novel reinforcement learning framework that learns a policy capable of generating significantly more diverse actions compared to existing methods. Our key idea is to train the policy with *random obstacle deployment*, pretending that non-existent obstacles exist. For instance, in Fig. 1 (*Upper-Middle*), we introduce a random obstacle that restricts the feasible action space. These obstacles force the policy to explore and learn a diverse set of actions that can avoid these obstacles, ultimately enabling it to generate a wide range of actions. Some care must be taken in designing the distribution of random obstacles to successfully learn a diverse policy. In our experiments, we demonstrate how these can be intuitively designed. We refer to our framework as **DIVERse** policy learning via random **O**bstacle deployment (**DIVO**).

To further enable zero-shot dynamic adaptation, we introduce two critical components: a state-dependent latent skill sampler and a motion predictor; see Fig. 1 (*Upper-Right*). The skill sampler generates multiple skill variables for each state. Since the conditional distribution $p(z|s)$ is complex, we employ a recent flow-based neural network model [15], [16]. The motion predictor then filters out unsafe actions by forecasting trajectories and identifying those that violate newly encountered obstacles (assuming we have a recognition module that provides the obstacles' geometries and positions). Together, these components ensure that the policy is not only diverse but also adaptable to dynamic and unpredictable environments, as shown in Fig. 1 (*Lower*).

Extensive simulation experiments demonstrate that our method generates significantly more diverse actions and achieves higher success rates in unseen environments with various obstacles (e.g., obstacle size, number of obstacles, dynamic obstacles) compared to existing methods. Additionally, when applied to a real-world robot for the Push-T task without fine-tuning, our policy shows a notably high success rate, despite some sim-to-real discrepancies.

II. RELATED WORKS

Our main objective is to learn an adaptive policy that can be used in environments not encountered during training, specifically those involving unforeseen obstacles. In this section, we summarize existing reinforcement learning approaches that can be used for this purpose.

One research direction aligned with our problem is the unsupervised skill discovery framework, which identifies diverse

skills without extrinsic task rewards. For instance, DIAYN [8], DADS [11] maximizes mutual information between state or state-action pair and skill variable, acquiring diverse skills. However, recent studies highlighted that maximizing mutual information may not fully cover the state space, leading some researchers to incorporate geometric properties to improve state space coverage [12], [13].

In scenarios where external task rewards are present, approaches such as LTD3 [7] and SMERL [1] attempt to maximize mutual information between state or state-action pair and skill variable, similar to unsupervised skill discovery techniques, while also maximizing the external rewards. However, these methods often yield less-than-desirable skill diversity due to the inherent limitations of mutual information. Our framework addresses this by generating diverse actions without relying on mutual information.

More recently, Cheng et al. [17] developed a framework for learning diverse policies by diversifying features of the trajectories, such as velocity direction, in quadruped robot navigation tasks. While effective, this approach relies on careful feature selection and additional network training, such as a success feature network, to achieve the desired diversity. In contrast, our method is simpler, requiring no extra network training, and the design of a random obstacle distribution is straightforward and intuitive.

Furthermore, unlike the aforementioned methods that keep the latent skill variable fixed throughout an episode, our approach allows the latent skill to vary within an episode, enabling adaptation to dynamically changing environments through a state-dependent latent skill sampler and a motion predictor.

Apart from our strategy of using diverse policies to adapt to unseen obstacles, some approaches focus on training end-to-end policies conditioned on input observations that contain obstacle information to generalize to unseen obstacles. For instance, random obstacle placement has been used as a domain randomization technique to train policies robust to various types of obstacles [18], [19]. However, such conditional policies (e.g., vision-conditioned policies) often struggle with out-of-distribution (OOD) obstacles not encountered during domain randomization. Training with an extremely wide range of obstacle variations is nearly infeasible or highly costly. In contrast, our model is trained with simple obstacle placements yet remains effective even for OOD obstacles.

III. DIVERSE POLICY LEARNING VIA RANDOM OBSTACLE DEPLOYMENT

We adopt standard notation from the deep reinforcement learning literature and formulate it as an optimization of cumulative discounted reward in a Markov Decision Process (MDP) $\mathcal{M}$, defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, \mu)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}(s_{t+1}|s_t, a_t)$ is the transition dynamics, $\mathcal{R}(s_t, a_t)$ is a reward function, γ is a discount factor, and μ is the initial state distribution. We consider a stochastic policy $\pi(a_t|s_t)$, a distribution over actions conditioned on the state. The expected discounted sum of rewards of a policy π is given by $\mathcal{R}(\pi) = \mathbb{E} \left[\sum_{t=1}^T \gamma^t \mathcal{R}(s_t, a_t) \right]$, where $s_0 \sim \mu, a_t \sim \pi(\cdot|s_t), s_{t+1} \sim \mathcal{P}(\cdot|s_t, a_t)$ for $t = 1, \dots, T$ with some terminal time length T . Then the goal of reinforcement learning is to find the optimal policy π^* that maximizes $\mathcal{R}(\pi)$.

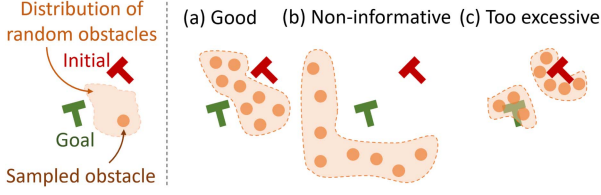


Fig. 2. Example distributions of random obstacles in the Push-T task.

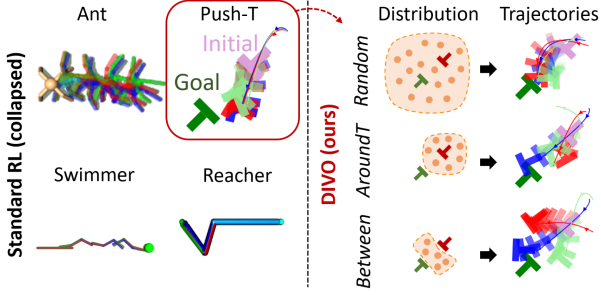


Fig. 3. Effect of obstacle distribution on policy diversity. *Left*: Training without obstacles leads to collapsed actions. *Right*: Training with different obstacle distributions (*Random*, *AroundT*, and *Between*) leads to different levels of trajectory diversity, where the *Between* produces the most diverse policy, as it encounters more informative obstacles during training.

A. Latent-Conditioned Policies

To enable diverse action generation, we consider a latent variable policy model. Define a latent skill space $\mathcal{Z}$ and a latent skill variable $z \in \mathcal{Z}$. Then we consider a distribution of actions conditioned on both state s and skill z , denoted by $\pi(a|s, z)$. We assume that the skill variable z has a distribution dependent on the state s , denoted by $p(z|s)$. Prior works [1], [7], [8] assume the distribution of z to be unconditional. However, this approach limits the overall expressiveness of the policy since a neural network model $\pi(a|s, z)$ alone has to handle all the complexity caused by the state dependency. Therefore, we adopt a state-dependent skill distribution.

Consequently, we consider the following form of a latent-conditioned stochastic policy:

$$\pi(a|s) = \int_{\mathcal{Z}} \pi(a|s, z) p(z|s) dz. \quad (1)$$

Designing the model in this way alone and applying standard reinforcement learning algorithms is not sufficient to learn a diverse policy. They often result in $\pi(a|s, z)$ collapsing – where different skill variables lead to the same action distribution –, generating only a single action; see Fig. 3 (*Left*). In the subsequent sections, we propose a novel algorithm that results in a diverse policy.

B. Learning Diverse Policies Via Random Obstacle Deployment

In this section, we introduce a method to train $\pi(a|s, z)$, where each z leads to a different action distribution. The first step is to design a *distribution of random obstacles*, where we represent using inequality constraints. Specifically, we define an obstacle as an inequality constraint $c_\omega(s) \leq 0$ parametrized by $\omega \in \Omega$ and a distribution of parameters conditioned on the initial state s_0 , denoted by $p(\omega|s_0)$. Then, in the beginning of an episode

Algorithm 1: TD3 with Random Obstacles.

```

Initialize policy network  $\pi_\phi$ , skill encoder  $E_\eta$ , critic
network  $Q_{\theta_1}, Q_{\theta_2}$ , replay buffer  $\mathcal{B}$ , terminal length  $T$ 
Initialize target networks
 $\theta'_1 \leftarrow \theta_1, \theta'_2 \leftarrow \theta_2, \phi' \leftarrow \phi, \eta' \leftarrow \eta$ 
for episode = 1, ...,  $M$  do
  Sample initial state  $s_0 \sim \mu$  and random obstacles
   $\omega \sim p(\omega|s_0)$ , done  $\leftarrow$  False, and  $t \leftarrow 1$ 
  while not done do
    Select action  $a = \pi_\phi(s, E_\eta(s, \omega))$  and observe reward
     $r$  and state  $s'$ .
    Store transition tuple with the obstacle parameter
     $(s, a, r, s', \omega)$  in  $\mathcal{B}$ 
    Sample mini-batch of  $N$  transitions  $(s, a, r, s', \omega)$ 
    from  $\mathcal{B}$ 
     $\tilde{a} \leftarrow \pi_{\phi'}(s', E_{\eta'}(s', \omega)), y \leftarrow$ 
     $r + \gamma \min_{i=1,2} Q_{\theta'_i}(s, \tilde{a}, \omega)$ 
    Update critics
     $\theta_i \leftarrow \operatorname{argmin}_{\theta_i} N^{-1} \sum (y - Q_{\theta_i}(s, a, \omega))^2$ 
    Find the minimum index  $i^* \leftarrow \operatorname{argmin}_{i=1,2} Q_{\theta_i}(s, a, \omega)$ 
    Update  $\phi, \eta$  using the policy gradient
     $N^{-1} \sum \nabla_a Q_{\theta_{i^*}}(s, a, \omega) \big|_{a=\pi_\phi(s, E_\eta(s))} \nabla_{(\phi, \eta)} \pi_\phi(s, E_\eta(s))$ 
    Update target networks by moving average:
     $(\theta'_i, \phi', \eta') \leftarrow \tau(\theta_i, \phi, \eta) + (1 - \tau)(\theta'_i, \phi', \eta')$ 
    if  $c_\omega(s) > 0$  or  $t = T$  or task-completed: done  $\leftarrow$  True
    else:  $t \leftarrow t + 1$ 
  end while
end for

```

with an initial state s_0 , we can sample a random obstacle via $\omega \sim p(\cdot|s_0)$. For example, revisit the push-T task in Fig. 2. We may consider a circular obstacle with a fixed radius and define its position parameter as $\omega \in \mathbb{R}^2$. And we can design $p(\omega|s_0)$ so that its samples lie around the initial T-block position as shown in Fig. 2.

During training, given a sampled obstacle, if state s violates $c_\omega(s) \leq 0$, the episode terminates and receives a significant negative reward. If a solution exists, we assume there exists a skill variable $z \in \mathcal{Z}$ for each state s such that $\pi(a|s, z)$ can complete the task without violating the constraint. And, for each ω and s , we aim for a different skill variable z to be learned.

When designing the distribution of obstacles, there are two key points to consider carefully. First, the obstacle should be informative, meaning that the solution with the obstacle should differ from the solution without the obstacles. Otherwise, the same skill will be learned for different ω . For instance, in the push-T task, the circular object should not be placed too far from the T-block's initial and goal poses as shown in Fig. 2(b), rather need to obstruct the space between these poses as shown in Fig. 2(a). Second, the obstacle should not be too excessive; without a feasible solution, an appropriate skill cannot be learned. For example, if the circular obstacles are positioned too close to either the initial or goal pose as shown in Fig. 2(c), this could result in an infeasible solution. If the obstacle distribution effectively meets the aforementioned two criteria, we can learn a policy $\pi(a|s, z)$ capable of generating diverse actions by adjusting z ; see Fig. 3 (*Right*).

Now we provide specific details of the training process. The reward function is modified to $\mathcal{R}(s, a, \omega) = \mathcal{R}(s, a) - k\mathbb{1}(C_\omega(s) > 0)$ with $k > 0$. We introduce a skill encoder

$E : \mathcal{S} \times \Omega \rightarrow \mathcal{Z}$, which will be simultaneously trained with $\pi(a|s, z)$. It is important to note that this encoder is introduced for training purposes but will not be used during inference. We adopt the actor-critic framework from standard model-free reinforcement learning algorithms, but the critic function is modified to take the obstacle parameter ω as an additional input, i.e., $Q : \mathcal{S} \times \mathcal{A} \times \Omega \rightarrow \mathbb{R}$. While our approach can be combined with any standard actor-critic reinforcement learning algorithm, in this paper, we focus on the Twin Delayed Deep Deterministic Policy Gradient algorithm (TD3) [20].

Throughout, we assume $\pi(a|s, z) = \delta(a - \pi(s, z))$ with a deterministic function $\pi(s, z)$, where δ is a dirac delta function. We use a neural network policy π_ϕ , skill encoder E_η , and two critic networks $Q_{\theta_1}, Q_{\theta_2}$ with network parameters $\phi, \eta, \theta_1, \theta_2$. We employ standard techniques used in actor-critic methods [21], [22], [23], [24], such as replay buffers, target networks, moving averages, and policy gradients with critics. However, our approach differs in that it involves sampling and incorporating random obstacles ω during training. The pseudocode is provided in Algorithm 1.

C. Learning Latent Skill Samplers and Flow Matching

In the previous section, we learned $\pi(a|s, z)$; however, not all skill variable $z \in \mathcal{Z}$ would lead to a valid action. In this section, we propose a method to learn a distribution of skills conditioned on the state $p(z|s)$ that results in valid actions. To train $p(z|s)$, we collect a dataset of (z, s) -pairs by using the trained encoder and policy as follows: $s_0 \sim \mu, \omega \sim p(\cdot|s_0)$, and $z_t = E_\eta(s_t, \omega)$, $a_t = \pi_\phi(s_t, z_t)$, $s_{t+1} \sim \mathcal{P}(\cdot|s_t, a_t)$ for $t = 0, \dots, T-1$. The conditional distribution $p(z|s)$ can be very complex and challenging to model with simple classical probabilistic approaches. To address this, we utilize continuous normalizing flow models and the flow matching algorithm [15], [16], which have recently proven effective in various robot learning contexts [6], [25], [26], [27], [28]. Given a prior distribution $z_0 \sim p(z_0)$ such as Gaussian, we use a neural network time-dependent velocity field $v_\alpha(t, z, s)$ parameterized by α to model $p(z|s)$ as the distribution of z_1 such that z_t is a solution trajectory of the ODE $\dot{z} = v_\alpha(t, z, s)$ over time $t \in [0, 1]$. Training v_α using the maximum likelihood method is time-consuming due to required simulations. Conversely, employing the recently developed flow matching algorithm [16], a simulation-free approach, allows us to find superior solutions much more rapidly.

IV. ZERO-SHOT DYNAMIC ADAPTATION

In the previous section, we introduced how to learn the policy $\pi_\phi(s, z)$ and latent skill sampler $p_\alpha(z|s)$. Therefore, given the current state s , we are now able to generate diverse actions by first sampling a latent skill z and then generating an action with π_ϕ . However, we note that, given a new environmental constraint, not all generated actions may satisfy this constraint. Therefore, when selecting an action from diverse candidates, we need to consider whether the motion generated by the selected action will satisfy the new constraint. In the following subsections, we first introduce a motion predictor (Section IV-A), which forecasts future trajectories. Then, in Section IV-B, we outline our dynamic skill adjustment method for adaptation to changing environments.

A. Motion Predictor

We introduce a *motion predictor* network designed to forecast state changes in response to an action input. In this context, “motion” represents the trajectory of a state over time. If an analytical model of the environment’s dynamics is available, it can be used to prevent unsafe actions. However, in many real-world scenarios, these dynamics are either unknown or challenging to model accurately (e.g., contact dynamics), necessitating the use of a learned motion predictor. Let s_t be the current state and a_t be the current action; denote the next state by s_{t+1} . In our approach, motion is defined as the sequence of states from s_t to s_{t+1} , denoted by $s_{t:t+1}$, e.g., with a sequence length H , $s_{t:t+1} := \left(s_{t+\frac{k}{H-1}} \right)_{k=0}^{H-1} \in \mathcal{S}^H$. When using the notation s for state instead of s_t (without t), we denote $s_{t:t+1}$ as s_{traj} . We train a neural network motion predictor that predicts a state trajectory $m_\beta : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}^H$ with parameters β , denoted by $s_{\text{traj}} = m_\beta(s, a)$. One may wonder why it is insufficient to learn the conventional transition dynamics $s_{t+1} = f(s_t, a_t)$. For example, in the Push-T task, the action a_t transitions the state from s_t to s_{t+1} , often covering a considerable distance. In such cases, intermediate states along $s_{t:t+1}$ may violate constraints. Thus, predicting the entire trajectory $s_{t:t+1}$ is necessary to ensure constraint compliance, highlighting the limitations of conventional transition dynamics modeling.

Given that the motion predictor effectively learns the environment’s dynamics, one might argue that trajectory optimization-based techniques, like Model Predictive Control (MPC), could solve the task while avoiding obstacles, potentially negating the need for policy learning. However, these methods face significant challenges, particularly the high computational cost in complex, high-dimensional environments. In contrast, our approach trains a policy that encodes a set of effective trajectories, allowing it to quickly select an appropriate one through sampling rather than solving an optimization problem on the fly. This enables faster, more efficient adaptation in dynamic and unpredictable environments, making it better suited for real-world applications.

B. Dynamic Skill Adjustment

In this section, we introduce how the policy π_ϕ , latent skill sampler v_α , and motion predictor m_β can be used together to adapt to dynamically changing unseen environments. Existing methods [1], [7], [8] select a skill variable based on the initial state s_0 and a new given constraint $c_{\text{new}}(s) \leq 0$, and do not change it within the episode. Therefore, they are difficult to apply in environments where constraints change dynamically. In contrast, we can resample the skill depending on the current state by the following procedure: (i) sample multiple skills using v_α , (ii) compute multiple actions using π_ϕ , (iii) predict motions using m_β , (iv) check which motions satisfy the current constraint c_{new} , and (v) obtain safe action candidates; the overall procedure is illustrated in Fig. 4.

Given multiple safe action candidates, we need to choose one to execute. For this purpose, we design a score function SCORE that outputs the preference level of each action based on its predicted motion s_{traj} , such as the negative distance from s_{traj} to a new obstacle. Then, among the safe action candidates, we can choose the one that leads to the maximum score. We can repeat this at an appropriate frequency and perform skill re-sampling

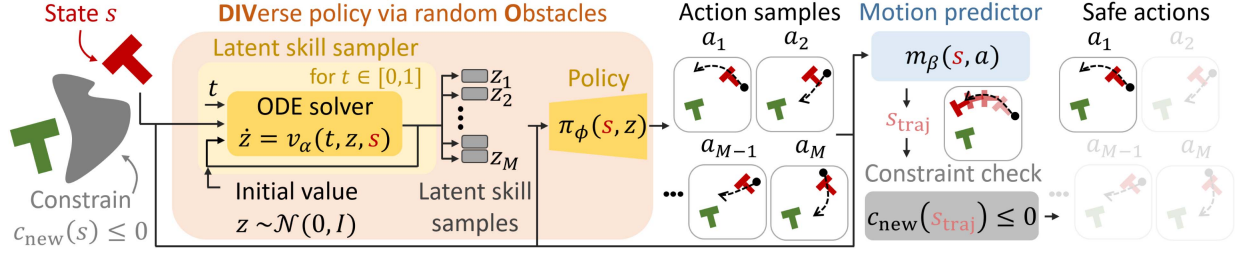


Fig. 4. Overview of the skill adjustment framework: Given a state s , we sample multiple skills and actions, predict motions s_{traj} , and obtain final safe action candidates by filtering out actions that produce s_{traj} violating the newly given constraint $c_{\text{new}}(s_{\text{traj}}) \leq 0$.

Algorithm 2: Dynamic Skill Adjustment Using Multi-Step Look-Ahead.

Policy π_ϕ , latent skill sampler p_α , motion predictor m_β , look-ahead size L , # of skill samples S , score function SCORE,
 initial state $s_0 \sim \mu$, and $\text{done} \leftarrow \text{False}$
while not done **do** at control frequency
 Observe the current state s and constraint $c(s) \leq 0$
 Let $\mathcal{I} \leftarrow$ an empty list
 for $i = 1, \dots, S$ **do** $s_0 \leftarrow s$
 for $l = 1, \dots, L$ **do**
 Sample a latent skill z_{il} from $p_\alpha(\cdot | s_{l-1})$
 $s_{l,\text{traj}} = m_\beta(s_{l-1}, \pi_\phi(s_{l-1}, z_{il}))$ and $s_l \leftarrow s_{l,\text{traj}}[-1]$
 end for
 Let $\text{score}_i \leftarrow \text{SCORE}(\text{concat}(s_{1,\text{traj}}; \dots; s_{L,\text{traj}}))$
 if $c(s_{l,\text{traj}}) \leq 0$ for $\forall l: \mathcal{I}.\text{append}(i)$
 end for
 if task-completed or $\mathcal{I}$ is empty: $\text{done} \leftarrow \text{True}$
 else: Select an $i^* = \arg \max_{i \in \mathcal{I}} \text{score}_i$ and perform an action $a \leftarrow \pi_\phi(s, z_{i^*1})$
end while

in a feedback manner. Therefore, our method is applicable even in dynamic environments.

Sometimes, if we predict only one step ahead and choose a latent skill and action, there may be no feasible actions available in the next state. In such cases, selecting skills and actions by considering multi-step futures can be a solution. Denoting the sampling through the ODE $\dot{z} = v_\alpha(t, z, s)$ by $z \sim p_\alpha(\cdot | s)$ and the multi-step size by L for the look-ahead, our dynamic skill adjustment algorithm is summarized in Algorithm 2.

V. EXPERIMENTAL RESULTS

In this section, we compare our **DIVERse** policy learning via random **Obstacle** deployment (**DIVO**) – denoted as **DIVO** + GMM and **DIVO** + FM, depending on the model used for the latent skill sampler – with existing mutual information-based diverse policy learning approaches: SAC+DIAYN [8], [23], SMERL [1], and LTD3 [7]. Additionally, we incorporate an end-to-end reinforcement learning (E2E) method for comparison, where the policies are conditioned on both the agent’s states and the obstacles’ states (including their positions and sizes), which we denote by E2E (w/ ground-truth state).¹ To ensure

generalization to obstacle sizes, this method is trained using domain randomization with random placements of obstacles of various sizes. In contrast, our methods are trained exclusively on small obstacles, as our algorithm is designed to generalize effectively to environments with unseen obstacles of varying sizes and shapes – a capability that the E2E method lacks.

In section V-A, we provide simulation studies with four environments – Push-T, Ant, Reacher, and Swimmer – and report diversity metrics and success rates in unseen environments. In Section V-B, we apply our policy trained in the simulation Push-T environment to the real world without re-training. As shown in Algorithm 2, zero-shot adaptation requires verifying whether predicted future states violate constraints. To enable this process, we assume the presence of a recognition module (e.g., a vision-based system) capable of detecting obstacles’ positions and geometries. This information is used to determine safe, feasible actions during evaluation. Specifically, for the real-world Push-T experiments, we use depth images to detect the positions of the T-shaped block and surrounding obstacles. This assumption is essential for ensuring the feasibility of generated actions in environments with varying constraints.

It’s important to note that during training, our policy was only exposed to environments with a single small obstacle. Despite this, in testing, the policy was evaluated in scenarios with entirely different obstacles that it had never encountered before, including multiple and dynamically changing obstacles. This highlights the robustness of our method in generalizing to new, unforeseen environments. Although our experiments focus specifically on obstacle avoidance tasks, we believe that these serve as a strong demonstration of the effectiveness of our approach for diverse policy learning. The ability to successfully navigate between obstacles of different shapes and sizes suggests that the policy is not only effective in obstacle-rich environments but also has the potential to generalize to other dynamic and complex scenarios.

A. Simulation Studies

Environments: We consider four RL environments as shown in Fig. 6 (Upper).

Push-T [4] requires pushing a T-shaped block to a desired pose. The state is represented as a 4-dimensional vector comprising the 2D coordinates of the block (x, y) and its rotation encoded as $(\cos \theta, \sin \theta)$. The action is an 8-dimensional vector representing four 2D control points in the workspace. These control points are used to generate the end-effector’s trajectory via cubic spline interpolation.

Ant [29] involves a four-legged robot with hinge joints, tasked with navigating to a target location. The state is a 39-dimensional

¹We excluded a baseline with a vision-conditioned policy – one that directly takes vision observations as input – since it failed to converge to a reasonable policy in most of our experimental settings, except for the Push-T task.

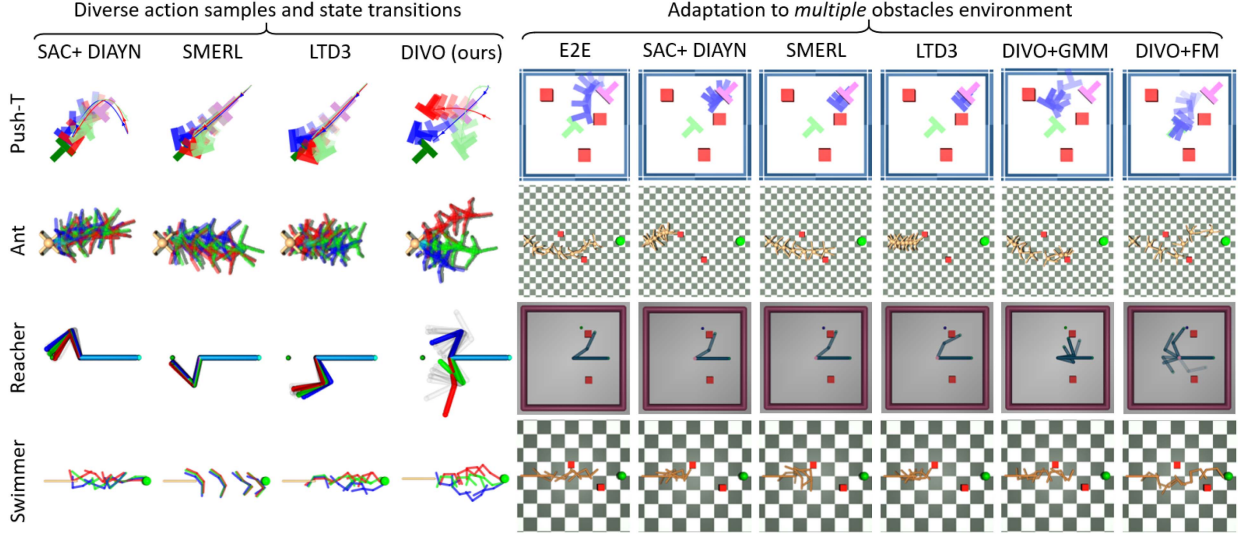


Fig. 5. *Left*: Diverse action samples and corresponding state transitions. For the Push-T, Ant, and Swimmer cases, state trajectories are visualized, and for the Reacher case, next state is visualized. *Right*: Policy adaptation to unseen environments with multiple obstacles.

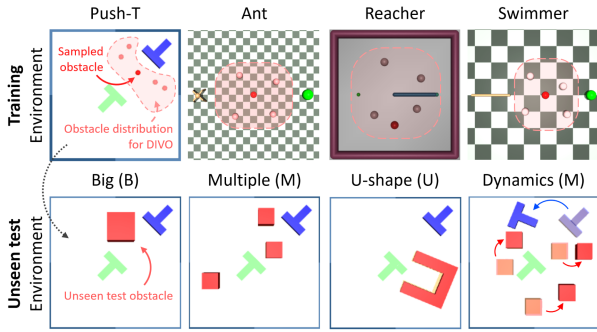


Fig. 6. *Upper*: Four RL environments: Push-T, Ant, Reacher, and Swimmer. Red obstacles represent the distributions of the random obstacles. *Lower*: Unseen test environments for Push-T.

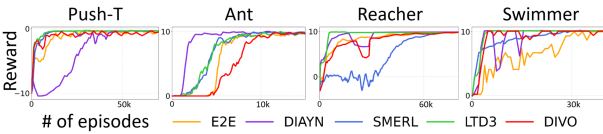


Fig. 7. Average reward per episode for different algorithms across all environments as a function of the number of training episodes.

vector that includes the positional data of the torso, joint angles, and their velocities. The action is an 8-dimensional vector representing torques applied to the hinge joints.

Reacher [29] is a 2-DoF robot arm whose end-effector is required to reach a target point. The state is 10-dimensional vector including joint angles, the angular velocity of each joint, the target coordinates, and the vector between the target and the end-effector. The action is an 8-dimensional vector, where the policy generates four sequential torques for each of the two joints.

Swimmer [29] is a three-link robot with two rotational joints, required to reach a target position by crawling like a snake. The state is a 10-dimensional vector capturing the x and y coordinates of the front tip, orientation of the front tip, joint

angles, and joint velocities. The action is an 8-dimensional vector consisting of four control points for torques. These control points are used to generate a torque trajectory through cubic spline interpolation which is then applied to the joints.

Training: For DIVO, we define random obstacles that limit the feasible action space, as described in Section III-B; see Fig. 6 (Upper). We sample a single obstacle for each episode during training. All algorithms use similarly sized policy network architectures, consisting of fully connected layers (FCNs) with ReLU activations, to eliminate potential effects caused by differences in network size. For diverse policy learning methods, we use a three-dimensional latent skill space, $z \in \mathbb{R}^3$, except for LTD3, which utilizes a hybrid latent space, $\mathbb{R}^3 \times \{0, 1\}^3$. The training process was conducted using a GeForce RTX 3090 GPU. Each algorithm was trained until the reward converged to a steady value, as shown in the training graphs in Fig 7.

Evaluation: We measure action diversities and success rates in unseen environments. The diversity is measured with the variance of action samples, averaged over states. The success rate is determined by whether the agent reaches the goal without colliding with any obstacles. To test adaptability, four unseen types of environments are considered; see Fig 6 (Lower): first, we randomly put a *Big (B)* obstacle, larger than the obstacle size used in DIVO training; second, we randomly put *Multiple (M)* obstacles (three for the Push-T task, and two each for the Ant, Reacher, and Swimmer tasks); third, we introduce a non-convex *U-shape (U)* obstacle, which is significantly larger than the *Big* obstacle and has a complex structure. And fourth, we put an obstacle *Dynamically (D)*. In the dynamic obstacle scenarios, for the Push-T task, we randomly relocate the positions of the three obstacles after each action is executed; for the Ant, Reacher, and Swimmer tasks, we suddenly put an obstacle that blocks the original path in the middle of the episode.

Table II shows the action diversity metrics, the variance of actions, where a higher value indicates better performance. In all examples, DIVO achieves significantly greater diversities. Fig. 5 (Left) visualizes state transitions caused by multiple action samples. In the Push-T, Ant, and Swimmer rows, state trajectories are visualized, and in the Reacher row, only the next

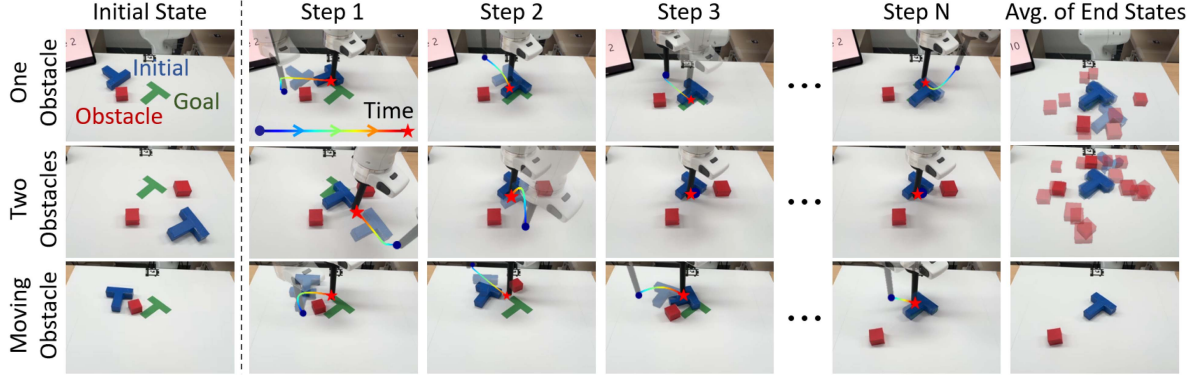


Fig. 8. Deployment of the trained policy in a real-world setting, where the end-effector path moves from the blue circle (●) to the red star (★). *Upper*: One obstacle case, *Middle*: Two obstacles case, and *Lower*: A moving obstacle case. Upper and middle rows of the final column show the averaged end states over 10 trials.

TABLE I
SUCCESS RATES IN UNSEEN STATIC ENVIRONMENTS WITH A BIG (B), MULTIPLE (M), AND U-SHAPE (U) OBJECT, AND DYNAMIC (D) ENVIRONMENTS

METHOD	Push-T	Ant	Reacher	Swimmer
	B / M / U / D	B / M / U / D	B / M / U / D	B / M / U / D
E2E (w/ ground-truth obstacle)	0.69 / 0.79 / 0.08 / 0.22	0.75 / 0.66 / 0.25 / 0.62	0.98 / 0.67 / 0.98 / 0.38	0.66 / 0.52 / 0.60 / 0.30
SAC+DIAYN	0.74 / 0.52 / 0.62 / 0.40	0.40 / 0.05 / 0.29 / 0.05	0.77 / 0.58 / 0.77 / 0.01	0.35 / 0.60 / 0.27 / 0.10
SMERL	0.47 / 0.42 / 0.31 / 0.29	0.49 / 0.11 / 0.30 / 0.13	0.75 / 0.55 / 0.76 / 0.01	0.00 / 0.20 / 0.01 / 0.00
LTD3	0.57 / 0.48 / 0.51 / 0.39	0.47 / 0.09 / 0.32 / 0.10	0.79 / 0.59 / 0.80 / 0.00	0.50 / 0.45 / 0.42 / 0.10
DIVO + GMM (ours)	0.84 / 0.70 / 0.75 / 0.19	0.13 / 0.06 / 0.06 / 0.00	0.90 / 0.77 / 0.98 / 0.47	0.14 / 0.50 / 0.03 / 0.01
DIVO + FM (ours)	0.98 / 0.97 / 0.91 / 0.95	0.88 / 0.81 / 0.66 / 0.82	0.98 / 0.88 / 0.98 / 0.78	0.85 / 0.80 / 0.73 / 0.45

TABLE II
EVALUATION METRICS OF ACTION DIVERSITY; THE HIGHER, THE BETTER

	Push-T	Ant	Reacher	Swimmer
SAC+DIAYN [8]	0.0338	0.6954	0.2265	0.3666
SMERL [1]	0.0103	0.0784	0.0044	0.0088
LTD3 [7]	0.0055	0.3324	0.2733	0.3421
DIVO (ours)	0.6250	1.4235	2.3798	2.1489

state is visualized. We can also qualitatively confirm that our method generates significantly more diverse actions that lead to diverse state transitions.

Table I shows that our method (DIVO + FM) achieves the highest success rates across all experiments. The end-to-end (E2E) method underperforms due to its limited generalization capabilities, even when trained with domain randomization. Similarly, existing diverse policy learning methods underperform due to insufficient action diversity. In particular, compared to existing diverse policy methods, the performance gap is most pronounced in dynamic obstacle scenarios. This is because these methods fail to adapt their latent skills dynamically, leading to collisions as they persist along their initial paths. Lastly, DIVO + GMM exhibits significantly lower success rates compared to DIVO + FM. This outcome highlights the Gaussian Mixture Model’s (GMM) inability to effectively capture the complexity of the state-dependent latent skill distribution. Fig. 5 (Right) demonstrates that only our DIVO + FM successfully adapts to unseen environments with multiple obstacles.

B. Real Robot Push-T Task

In this section, we apply our DIVO policy, trained in the simulation Push-T environment, to a real-world setting without

fine-tuning and verify zero-shot sim-to-real transfer. We construct the real-world setting to be as similar as possible to the simulation environment.

We deploy the policy in three unseen environments: one with a single obstacle, another with two obstacles, and the last with a moving obstacle. Throughout, we intentionally place obstacles that block certain areas between the initial T-block state and the goal state, thereby restricting the feasible action space. In the last experiment, after each action, we reposition the obstacle to block the path to the goal three times. Finally, we move the obstacle far enough away to allow the agent to reach the goal.

For the experiments with one and two obstacles, we report the success rates measured over 10 trials. The robot achieves a 90% success rate in both tasks. We found that the failures are due to the sim-to-real gap in the motion predictor, which increases as the T-block moves over greater distances.

Fig. 8 illustrates how the real robot, employing our policy, pushes the T-block. For the one obstacle case, as shown in the upper row, a motion that pushes the T-block above the obstacle is generated. In the middle row, the robot successfully pushes the block through the narrow space between two obstacles without collision. Lastly, in the lower row, we can observe that the policy outputs actions that push the block upwards if the obstacle is positioned below, and downwards if it is above, successfully adapting to the dynamic environment changes.

VI. CONCLUSION

We have proposed a novel diverse policy learning framework that can be used with any model-free actor-critic reinforcement learning algorithms. By introducing random obstacle deployment during training, we encourage policy exploration and the learning of diverse actions. Our approach also includes a latent

skill sampler, which generates skill variables from complex conditional distributions, and a motion predictor, which forecasts whether actions will satisfy environmental constraints. Using four simulation environments, we have confirmed that our model can produce much more diverse actions compared to existing state-of-the-art methods, leading to superior performance in unseen environments with both static and dynamic obstacles. Additionally, we have demonstrated the applicability of our model in the real-world Push-T task, achieving zero-shot sim-to-real transfer without further training.

Although our method can learn policies that generate significantly more diverse actions compared to existing methods, the action diversity of our approach heavily depends on the choice of random obstacle distributions. In this work, the distributions are manually designed for each task, and while they can be intuitively designed to some extent, there is no guarantee that this approach is suitable for arbitrary tasks. Therefore, developing an algorithm to automatically generate or adapt imaginary obstacles during training, rather than relying on manually designed random obstacles, represents an intriguing direction for future research.

Taking it a step further, a promising future research direction is to extend from random obstacles to random constraints to learn policies capable of generating even more diverse actions. Currently, as we only consider obstacles, the diversity achieved is primarily at the kinematic path level. This is a limitation compared to mutual information-based methods, which can produce more diverse behaviors beyond the kinematic level (e.g., for the ant example, various locomotion styles like running, walking, or rolling). Our framework has the potential to address this by introducing random constraints – instead of random obstacles – such as velocity limits or torque limits that enable the training of policies capable of generating a broader range of actions.

REFERENCES

- [1] S. Kumar, A. Kumar, S. Levine, and C. Finn, “One solution is not all you need: Few-shot extrapolation via structured MaxEnt RL,” *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 8198–8210, 2020.
- [2] B. Lee, Y. Lee, S. Kim, M. Son, and F. C. Park, “Equivariant motion manifold primitives,” in *Proc. 7th Annu. Conf. Robot Learn.*, 2023, vol. 229, pp. 1199–1221.
- [3] Y. Lee, “MMP++: Motion manifold primitives with parametric curve models,” *IEEE Trans. Robot.*, vol. 40, pp. 3950–3963, 2024.
- [4] C. Chi et al., “Diffusion policy: Visuomotor policy learning via action diffusion,” *Int. J. Robot. Res.*, 2023, Art. no. 02783649241273668.
- [5] M. Noseworthy, R. Paul, S. Roy, D. Park, and N. Roy, “Task-conditioned variational autoencoders for learning movement primitives,” in *Proc. Conf. Robot Learn.*, 2020, pp. 933–944.
- [6] Y. Lee, B. Lee, S. Kim, and F. C. Park, “Motion manifold flow primitives for task-conditioned trajectory generation under complex task-motion dependencies,” 2024, *arXiv:2407.19681*.
- [7] T. Osa, V. Tangkaratt, and M. Sugiyama, “Discovering diverse solutions in deep reinforcement learning by maximizing state–action-based mutual information,” *Neural Netw.*, vol. 152, pp. 90–104, 2022.
- [8] B. Eysenbach, A. Gupta, J. Ibarz, and S. Levine, “Diversity is all you need: Learning skills without a reward function,” 2018, *arXiv:1802.06070*.
- [9] D. Strouse, K. Baumli, D. Warde-Farley, V. Mnih, and S. Hansen, “Learning more skills through optimistic exploration,” 2021, *arXiv:2107.14226*.
- [10] J. Parker-Holder, A. Pacchiano, K. M. Choromanski, and S. J. Roberts, “Effective diversity in population based reinforcement learning,” *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 18050–18062, 2020.
- [11] A. Sharma, S. Gu, S. Levine, V. Kumar, and K. Hausman, “Dynamics-aware unsupervised discovery of skills,” 2019, *arXiv:1907.01657*.
- [12] S. Park, O. Rybkin, and S. Levine, “Metra: Scalable unsupervised RL with metric-aware abstraction,” 2023, *arXiv:2310.08887*.
- [13] S. Park, J. Choi, J. Kim, H. Lee, and G. Kim, “Lipschitz-constrained unsupervised skill discovery,” in *Proc. Int. Conf. Learn. Representations*, 2021.
- [14] S. Park, K. Lee, Y. Lee, and P. Abbeel, “Controllability-aware unsupervised skill discovery,” 2023, *arXiv:2302.05103*.
- [15] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, “Neural ordinary differential equations,” in *Proc. 32nd Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 6572–6583.
- [16] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” 2022, *arXiv:2210.02747*.
- [17] J. Cheng, M. Vlastelica, P. Kolev, C. Li, and G. Martius, “Learning diverse skills for local navigation under multi-constraint optimality,” in *Proc. 2024 IEEE Int. Conf. Robot. Automat.*, 2024, pp. 5083–5089.
- [18] F. Zhong, P. Sun, W. Luo, T. Yan, and Y. Wang, “AD-VAT+: An asymmetric dueling mechanism for learning and understanding visual active tracking,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 5, pp. 1467–1482, May 2021.
- [19] S. Pouyanfar, M. Saleem, N. George, and S.-C. Chen, “Roads: Randomization for obstacle avoidance and driving in simulation,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, 2019, pp. 1267–1276.
- [20] S. Fujimoto, H. Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1587–1596.
- [21] T. P. Lillicrap et al., “Continuous control with deep reinforcement learning,” 2015, *arXiv:1509.02971*.
- [22] V. Mnih et al., “Asynchronous methods for deep reinforcement learning,” in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1928–1937.
- [23] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
- [24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” 2017, *arXiv:1707.06347*.
- [25] E. Chisari, N. Heppert, M. Argus, T. Welschhold, T. Brox, and A. Valada, “Learning robotic manipulation policies from point clouds with conditional flow matching,” 2024, *arXiv:2409.07343*.
- [26] Y. Lee, “Trajectory manifold optimization for fast and adaptive kinodynamic motion planning,” 2024, *arXiv:2410.12193*.
- [27] B. Lim, J. Kim, J. Kim, Y. Lee, and F. C. Park, “Equigraspflow: Se (3)-equivariant 6-DoF grasp pose generative flows,” in *Proc. 8th Annu. Conf. Robot Learn.*, 2024.
- [28] F. Zhang and M. Gienger, “Robot manipulation with flow matching,” in *Proc. CoRL 2024 Workshop Mastering Robot Manipulation World Abundant Data*, 2024.
- [29] G. Brockman et al., “OpenAI gym,” 2016, *arXiv:1606.01540*.